CECS 326 – Operating Systems

Homework Assignment 1

Due Date: **TBD**

Objective

The objective of this assignment is to gain hands-on experience with process creation and synchronization in Linux using the `fork()` and `wait()` system calls.

Students will implement a multi-process program that creates a specific process hierarchy and correctly synchronizes parent and child processes.

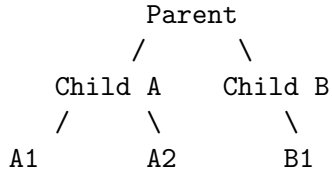
Learning Outcomes

After completing this assignment, students should be able to:

- Understand parent and child process relationships.
- Create multiple processes using `fork()`.
- Identify process IDs (PID) and parent process IDs (PPID).
- Synchronize processes using `wait()` or `waitpid()`.
- Understand process termination and execution order.

Required Process Tree

Your program must create the following process hierarchy.



The program should create exactly six processes:

- Parent
- Child A
- Child B
- Grandchild A1
- Grandchild A2
- Grandchild B1

Programming Requirements

Part 1 – Process Creation (25 points)

Create the exact process hierarchy shown above using the `fork()` system call.

Each process must display:

- Process name
- PID
- Parent PID (PPID)

Example:

```
Process: Child A
PID: 5342
Parent PID: 5331
```

Part 2 – Simulated Work (20 points)

Each process should perform independent work.

For example:

```
Child A
Working...
Step 1
Step 2
Step 3
Finished
```

Use

```
sleep(1);
```

between each step.

Part 3 – Process Synchronization (30 points)

Use `wait()` or `waitpid()` correctly.

The synchronization requirements are:

- Child A waits for A1 and A2.
- Child B waits for B1.
- Parent waits for Child A and Child B.
- The Parent process must terminate last.

Part 4 – Final Output (15 points)

After all child processes have completed, only the Parent process should print:

```
All child processes completed successfully.
```

Restrictions

You may **NOT** use:

- C++ threads (`std::thread`)
- `system()`
- `exec()` family of system calls
- Global variables for synchronization

You **MUST** use:

- `fork()`
- `wait()` or `waitpid()`
- Linux
- C++
- g++

Deliverables

Submit a single ZIP file named exactly as follows:

`lastname_HW01.zip`

Inside the ZIP file include:

- `process_tree.cpp`
- `Makefile`
- Screenshot(s) showing successful compilation and execution

Important:
Any submission that does not follow the required naming convention or required file structure will receive deductions according to the grading rubric.

Grading Rubric

Criterion	Points
Correct process hierarchy (exactly six processes created)	25
Correct implementation of `fork()`	10
Correct implementation of `wait()` / synchronization	20
Correct display of PID and PPID for every process	10
Each process performs independent work correctly	10
Parent terminates last and prints final completion message	15
Program compiles successfully using g++ without modification	5
Code readability, indentation, comments, and organization	5
Total	**100**

Submission Penalties

The following deductions apply in addition to the grading rubric.

Violation	Deduction
ZIP file name does not follow `lastname_HW01.zip` format	–5
Source file missing	–20
Makefile missing	–10
Screenshot missing	–5
Program does not compile	Up to –40
Program crashes during execution	Up to –30
Submitted files not inside a ZIP archive	–10
Incorrect or extra files causing grading issues	Up to –5

Reflection Questions (Not Graded)

The following questions are provided to help you deepen your understanding of process management. You are encouraged to research and understand these concepts independently.

These topics may appear on quizzes and exams.

1. What is the difference between a process and a program?
2. What is the difference between a parent process and a child process?
3. What does the `fork()` system call return in the parent process? In the child process?
4. Why is `wait()` necessary?

5. What happens if the parent process exits before its children?
6. What is a zombie process?
7. What is an orphan process?
8. What information is stored in a Process Control Block (PCB)?
9. What is the difference between a process ID (PID) and a parent process ID (PPID)?
10. What would happen if every process created another child without terminating properly?

Academic Integrity

All submitted work must be your own. You may discuss concepts with classmates, but the implementation, debugging, and submitted source code must be completed individually.

Students are responsible for ensuring that their programs compile and execute correctly before submission.